

MSDM5004

Numerical Methods and Modeling in Science
Spring 2026

Lecture 7

Prof Yang Xiang

Hong Kong University of Science and Technology

Chapter 8

Numerical Differentiation

1. Numerical differentiation

Review: The derivative of the function f at x_0 is

$$f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h}.$$

An approximation of the derivative

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}$$

where h is a small number.

Finite difference

Error analysis

$$g(x) = O(h(x)), \quad x \rightarrow x_0$$

$$\iff |g(x)| \leq M|h(x)|, \text{ near } x_0,$$

for some constant M .

Using Taylor expansion

$$f(x_0 + h) = f(x_0) + f'(x_0)h + \frac{1}{2}f''(\xi)h^2,$$

where ξ is between x_0 and $x_0 + h$.

We have

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2}f''(\xi).$$

The error of this approximation is $\frac{1}{2}f''(\xi)h$, which is bounded by $\frac{M}{2}h$ with M being the bound of $f''(x)$ over the interval $[x_0, x_0 + h]$. The error is $O(h)$. First order method.

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}$$

forward-difference formula if $h > 0$

backward-difference formula if $h < 0$.

An example

Use the forward-difference formula to approximate the derivative of $f(x) = \ln x$ at $x_0 = 1.8$ using $h = 0.1$, $h = 0.05$, and $h = 0.01$

Solution The forward-difference formula

$$f'(1.8) \approx \frac{f(1.8 + h) - f(1.8)}{h}$$

h	$\frac{f(1.8 + h) - f(1.8)}{h}$	$\frac{ h }{2(1.8)^2}$ (bound of error)
0.1	0.5406722	0.0154321
0.05	0.5479795	0.0077160
0.01	0.5540180	0.0015432

Note: The exact value of $f'(1.8) = \frac{1}{1.8} = 0.55555 \dots$

Central difference formula

$$f'(x_0) \approx \frac{1}{2h} [f(x_0 + h) - f(x_0 - h)]$$

Using Taylor expansion

$$f'(x_0) = \frac{1}{2h} [f(x_0 + h) - f(x_0 - h)] - \frac{h^2}{6} f^{(3)}(\xi_1),$$

where ξ_1 lies between $x_0 - h$ and $x_0 + h$.

The error is $O(h^2)$. Second order method.

Second Derivative

$$f''(x_0) \approx \frac{1}{h^2} [f(x_0 - h) - 2f(x_0) + f(x_0 + h)] .$$

Central difference formula

Using Taylor expansion

$$f''(x_0) = \frac{1}{h^2} [f(x_0 - h) - 2f(x_0) + f(x_0 + h)] - \frac{h^2}{12} f^{(4)}(\xi),$$

for some ξ , where $x_0 - h < \xi < x_0 + h$.

The error is $O(h^2)$. Second order method.

Constructing finite difference schemes using Taylor expansion

e.g. Construct a finite difference scheme for the derivative of $f(x)$ at the point x_0 , using values of $f(x_0)$, $f(x_0 + h)$, $f(x_0 + 2h)$.

Using Taylor expansion

$$f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{h^2}{2}f''(x_0) + \frac{h^3}{3!}f'''(x_0) + \dots$$

$$f(x_0 + 2h) = f(x_0) + 2hf'(x_0) + \frac{(2h)^2}{2}f''(x_0) + \frac{(2h)^3}{3!}f'''(x_0) + \dots$$

Assume

$$f'(x_0) = a_0 f(x_0) + a_1 f(x_0 + h) + a_2 f(x_0 + 2h) + O(?)$$

$$\text{Coefficients of } f(x_0): \quad a_0 + a_1 + a_2 = 0. \quad (1)$$

$$\text{Coefficients of } f'(x_0): \quad a_1 h + a_2 2h = 1. \quad (2)$$

$$\text{Coefficients of } f''(x_0): \quad a_1 \frac{h^2}{2} + a_2 \frac{(2h)^2}{2} = 0. \quad (3)$$

$$f(x_0 + h) = f(x_0) + h f'(x_0) + \frac{h^2}{2} f''(x_0) + \frac{h^3}{3!} f'''(x_0) + \dots$$

$$f(x_0 + 2h) = f(x_0) + 2h f'(x_0) + \frac{(2h)^2}{2} f''(x_0) + \frac{(2h)^3}{3!} f'''(x_0) + \dots$$

Solving this linear system,

$$a_0 = -\frac{3}{2h}, \quad a_1 = \frac{2}{h}, \quad a_2 = -\frac{1}{2h}.$$

The numerical scheme is

$$f'(x_0) \approx \frac{-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)}{2h}.$$

The error is $O(h^2)$.

Check order of a numerical scheme

Compute derivative of $f(x) = \ln x$ at $x_0 = 1.8$.

Numerical approximation $\phi_h(x_0) = \frac{f(x_0 + h) - f(x_0)}{h}$

$$\phi_h(x_0) = f'(x_0) + \frac{h}{2}f''(\xi), \quad \xi \in [x_0, x_0 + h]$$

$$= f'(x_0) + \frac{h}{2}f''(x_0) + O(h^2).$$

$$f''(\xi) = f''(x_0) + f'''(\xi_1)(\xi - x_0) = f''(x_0) + O(h)$$

$$\phi_h = f'(x_0) + h \cdot \frac{1}{2} f''(x_0) + O(h^2)$$

$$\phi_{\frac{h}{2}} = f'(x_0) + \frac{h}{2} \cdot \frac{1}{2} f''(x_0) + O(h^2)$$

$$\phi_{\frac{h}{4}} = f'(x_0) + \frac{h}{4} \cdot \frac{1}{2} f''(x_0) + O(h^2)$$

$$\frac{\phi_h - \phi_{\frac{h}{2}}}{\phi_{\frac{h}{2}} - \phi_{\frac{h}{4}}} \approx \frac{\left(h - \frac{h}{2}\right) \frac{1}{2} f''(x_0)}{\left(\frac{h}{2} - \frac{h}{4}\right) \frac{1}{2} f''(x_0)} = 2^1$$

“1” is the order of accuracy

Matlab code

```
x0=1.8;  
h=0.1;  
n=10;  
format long  
for i=1:n  
    phi(i)=(log(x0+h)-log(x0))/h;  
    h=h/2;  
end;  
for i=1:(n-2)  
    r=(phi(i)-phi(i+1))/(phi(i+1)-phi(i+2))  
end;  
phi(n)  
  
r = 1.946839686322930  
r = 1.972834761917758  
r = 1.986266023312467  
r = 1.993094509786362  
r = 1.996537545297094  
r = 1.998266333041434  
r = 1.999132550471774  
r = 1.999566142373304  
  
ans =  
  
0.555525416917817
```

Differentiation via polynomial interpolation

suppose that $\{x_0, x_1, \dots, x_n\}$ are $(n + 1)$ distinct numbers in some interval I and that $f \in C^{n+1}(I)$.

$f(x) \approx P_n(x)$ Lagrange polynomial of degree n

$$P_n(x) = \sum_{k=0}^n f(x_k) L_k(x)$$

$$L_k(x) = \frac{(x - x_0) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}.$$

$$f'(x) \approx P'_n(x)$$

Polynomial interpolation with error is

$$f(x) = \sum_{k=0}^n f(x_k) L_k(x) + \frac{(x - x_0) \cdots (x - x_n)}{(n+1)!} f^{(n+1)}(\xi(x)),$$

for some $\xi(x)$ in I , where $L_k(x)$ denotes the k th Lagrange coefficient polynomial for f at $x_0, x_1, \dots, x_n$.

Differentiating this expression gives

$$f'(x) = \sum_{k=0}^n f(x_k) L'_k(x) + D_x \left[\frac{(x - x_0) \cdots (x - x_n)}{(n+1)!} \right] f^{(n+1)}(\xi(x)) \\ + \frac{(x - x_0) \cdots (x - x_n)}{(n+1)!} D_x [f^{(n+1)}(\xi(x))].$$

$$D_x \Longleftrightarrow \frac{d}{dx}$$

When x is one of the numbers x_j ,

$$f'(x_j) = \sum_{k=0}^n f(x_k) L'_k(x_j) + \frac{f^{(n+1)}(\xi(x_j))}{(n+1)!} \prod_{\substack{k=0 \\ k \neq j}}^n (x_j - x_k),$$

$(n+1)$ -point formula to approximate $f'(x_j)$

All the formulas shown previously can also be obtained using this approach.

Automatic differentiation in neural network approximation

Neural network approximation of $f(\mathbf{x})$

$$f_{\boldsymbol{\theta}}(\mathbf{x}) \approx f(\mathbf{x})$$

Two layer neural network

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{j=1}^M a_j \sigma(\mathbf{w}_j \cdot \mathbf{x} + b_j)$$

$\boldsymbol{\theta}$ denotes all the parameters $\{(a_j, \mathbf{w}_j, b_j)\}_{j=1}^M$ $\sigma(\cdot)$ is the activation function

Automatic differentiation

$$\frac{\partial f}{\partial x_i} \approx \frac{\partial f_{\boldsymbol{\theta}}}{\partial x_i} = \sum_{j=1}^M a_j \sigma'(\mathbf{w}_j \cdot \mathbf{x} + b_j) w_{ij}$$

Chapter 9

Numerical solution of ordinary differential equations (ODEs)

I. Introduction

Initial value problem of ODE

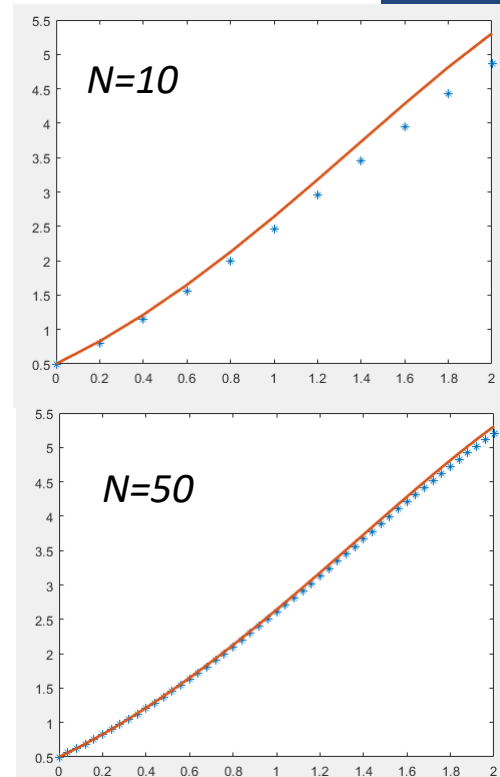
$$\frac{dy}{dt} = f(t, y), \quad a \leq t \leq b, \quad y(a) = \alpha$$

Approximation to y will be generated at discrete points, called **mesh points**, in the interval $[a, b]$.

$$t_i = a + ih, \quad \text{for each } i = 0, 1, 2, \dots, N.$$

$$y_0 = y(t_0)$$

$$y_i \approx y(t_i), \quad i = 1, 2, \dots, N$$



Idea 1

$$y'(t_i) = f(t_i, y(t_i))$$

Approximating the **derivative** by some finite difference scheme

$$y'(t_i) \approx \frac{y(t_{i+1}) - y(t_i)}{h}$$

Or

$$y'(t_{i+1}) \approx \frac{y(t_{i+1}) - y(t_i)}{h}$$

Idea 2

$$y'(t) = f(t, y(t)), t \in [t_i, t_{i+1}] :$$

$$\implies y(t_{i+1}) = y(t_i) + \int_{t_i}^{t_{i+1}} f(s, y(s)) ds$$

Approximating the **integral** using some numerical method

e.g., using trapezoidal rule to approximate the integral

$$y(t_{i+1}) \approx y(t_i) + \frac{h}{2} [f(t_i, y(t_i)) + f(t_{i+1}, y(t_{i+1}))]$$

2. Euler method

$$y_{i+1} = y_i + hf(t_i, y_i) \quad i = 1, 2, \dots, N$$

Sometimes, it also called forward Euler method.

Idea of derivation

$$\frac{y(t_{i+1}) - y(t_i)}{h} \approx y'(t_i) = f(t_i, y(t_i))$$

first order approximation

$$y_i \approx y(t_i) \quad i = 1, 2, \dots, N$$

A linear equation

$$y' = \lambda y \quad \lambda \text{ constant}$$

Using Euler method,

$$y_{i+1} = y_i + h\lambda y_i = (1 + h\lambda)y_i$$

$$y_i = (1 + h\lambda)^i y_0$$

Note: the exact solution of this ODE is $y(t) = e^{\lambda t} y(t_0)$

For a fixed time T , the numerical solution is

$$(1 + h\lambda)^{T/h} y_0 \rightarrow e^{\lambda T} y_0, \quad h \rightarrow 0$$
$$(N \rightarrow +\infty)$$

$$\left(1 + \frac{1}{x}\right)^x \rightarrow e, \quad x \rightarrow +\infty$$

Local error

Using Taylor expansion, the exact solution satisfies

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(\xi), \quad \xi \in [t_i, t_{i+1}]$$

The numerical solution using Euler method satisfies

$$y_{i+1} = y(t_i) + hy'(t_i) \quad \text{when } y_i = y(t_i)$$

Taking difference, the local error

$$e_{i+1} = y(t_{i+1}) - y_{i+1} = \frac{h^2}{2}y''(\xi) = O(h^2)$$

Global error

$$y'(t_i) = f(t_i, y(t_i))$$

$$|y(t_i) - y_i| \leq \frac{hM}{2L} [e^{L(t_i-a)} - 1]$$

$$O(h) \quad \text{error}$$

where $|y''(t)| \leq M$, for all $t \in [a, b]$

$$|f(t, y_1) - f(t, y_2)| \leq L|y_1 - y_2| \quad \text{Lipschitz condition}$$

Idea of the proof

$$y(t_{i+1}) = y(t_i) + hf(t_i, y(t_i)) + \frac{h^2}{2}y''(\xi)$$

$$y_{i+1} = y_i + hf(t_i, y_i)$$

$$y(t_{i+1}) - y_{i+1} = y(t_i) - y_i + h[f(t_i, y(t_i)) - f(t_i, y_i)] + \frac{h^2}{2}y''(\xi)$$

$$y(t_{i+1}) - y_{i+1} = y(t_i) - y_i + h[f(t_i, y(t_i)) - f(t_i, y_i)] + \frac{h^2}{2}y''(\xi)$$

Further using the Lipschitz condition

$$|f(t_i, y(t_i)) - f(t_i, y_i)| \leq L|y(t_i) - y_i|,$$

we have

$$|y(t_{i+1}) - y_{i+1}| \leq (1 + hL)|y(t_i) - y_i| + \frac{M}{2}h^2.$$

It can be proved that

$$|y(t_i) - y_i| \leq \frac{hM}{2L} [e^{L(t_i-a)} - 1]$$

Note that $t_i = ih + a$.

An example

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5.$$

With $N = 10$ we have $h = 0.2$, $t_i = 0.2i$, $y_0 = 0.5$

Using Euler method,

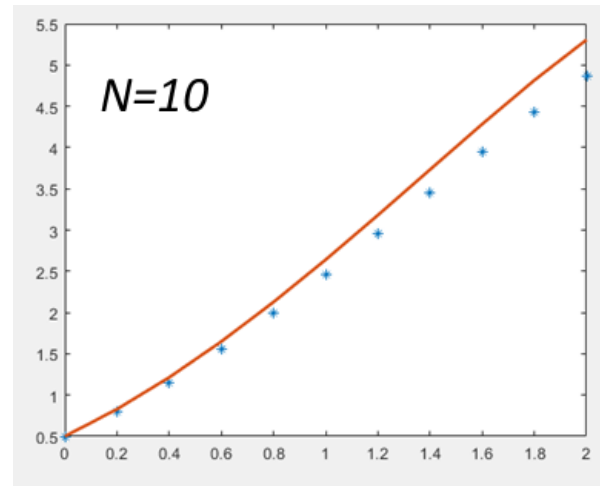
$$y_{i+1} = y_i + h(y_i - t_i^2 + 1)$$

$$y_1 = y_0 + h(y_0 - t_0^2 + 1) = 0.5 + 0.2(0.5 - 0^2 + 1) = 0.8$$

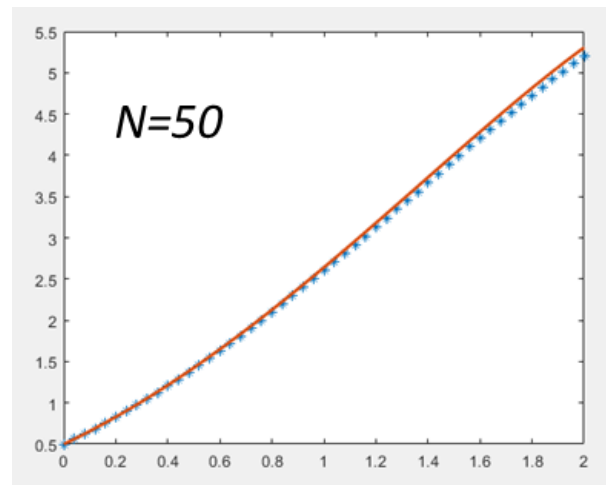
$$y_2 = y_1 + h(y_1 - t_1^2 + 1) = 0.8 + 0.2(0.8 - (0.2)^2 + 1) = 1.152$$

...

t_i	y_i
0.0	0.5000000
0.2	0.8000000
0.4	1.1520000
0.6	1.5504000
0.8	1.9884800
1.0	2.4581760
1.2	2.9498112
1.4	3.4517734
1.6	3.9501281
1.8	4.4281538
2.0	4.8657845



$N=50$



MATLAB code

```

n=10;
a=0;
b=2;
t=linspace(a,b,n+1);
y=zeros(n+1,1);
dt=(b-a)/n;
y(1)=0.5;
for i=1:n
    y(i+1)=y(i)+(y(i)-t(i)^2+1)*dt;
end;
plot(t,y,'*')

```

